SVEUČILIŠTE U ZAGREBU
FAKULTET ORGANIZACIJE I INFORMATIKE
VARAŽDIN

Gabriela Perković, Mia Šižgorić

ISPITIVANJE UTJECAJA RAZLIČITIH ARHITEKTURA NEURONSKE MREŽE NA KVALITETU MODELA KLASIFIKACIJE SLIKA U DUBOKOM UČENJU

PROJEKT

Varaždin, 2026.

SVEUČILIŠTE U ZAGREBU

FAKULTET ORGANIZACIJE I INFORMATIKE

V A R A Ž D I N

Gabriela Perković, Mia Šižgorić

Matični broj: 0016161159, 0016160216

Studij: Informacijski i poslovni sustavi

ISPITIVANJE UTJECAJA RAZLIČITIH ARHITEKTURA NEURONSKE MREŽE NA KVALITETU MODELA KLASIFIKACIJE SLIKA U DUBOKOM UČENJU

PROJEKT

Mentor:

doc. dr. sc. Bogdan Okreša Đurić

Varaždin, siječanj 2026.

Gabriela Perković, Mia Šižgorić

Izjava o izvornosti

Izjavljujem da je ovaj projekt izvorni rezultat mojeg rada te da se u izradi istoga nisam koristio drugim izvorima osim onima koji su u njemu navedeni. Za izradu rada su korištene etički prikladne i prihvatljive metode i tehnike rada.

Autorice potvrdile prihvaćanjem odredbi u sustavu FOI Radovi

Sažetak

Ovaj rad bavi se problematikom klasifikacije slika primjenom konvolucijskih neuronskih mreža na skupu podataka CIFAR-10. Svrha rada je provesti analizu između osnovnog CNN modela izrađenog tijekom laboratorijskih vježbi te naprednih arhitektura MobileNetV2 i EfficientNetB1, koristeći tehniku prijenosa učenja. Teorijsko polazište rada obuhvaća pregled razvoja CNN arhitektura, s fokusom na optimizacijske tehnike poput dubinski razdvojenih konvolucija i složenog skaliranja. Rad ispituje utjecaj različitih parametara predobrade podataka na konačnu točnost modela. Glavni dio istraživanja demonstrira da izravna primjena pretreniranih modela na slike niske rezolucije (32x32 piksela) nije učinkovita, što je potvrđeno nužnim povećanjem rezolucije na 96x96 piksela za postizanje optimalnih performansi MobileNetV2 modela. Nadalje, na primjeru EfficientNetB1 arhitekture dokazana je kritična važnost usklađivanja statističke distribucije podataka; usporedba korištenja namjenske `preprocess_input` funkcije naspram ručne normalizacije pokazala je da neadekvatna priprema podataka dovodi do potpunog neuspjeha procesa učenja.

Ključne riječi: konvolucijske neuronske mreže; klasifikacija slika; CIFAR-10; prijenos učenja; MobileNetV2; EfficientNetB1; normalizacija podataka; skaliranje rezolucije.

Sadržaj

1. Uvod 1

2. Definicije neuronskih mreža 2

3. Arhitekture neuronske mreže 3
3.1. LeNet 3
3.2. MobileNet 3
3.3. EfficientNet 4

4. Opis implementacije 5
4.1. Rješenje sa laboratorijskih vježbi 5
4.2. MobileNetV2 6
4.3. EfficientNetB1 8

5. Prikaz rada 11
5.1. Kod sa laboratorijskih vježbi 11
5.2. MobileNetV2 12
5.3. EfficientNetB1 13

6. Kritički osvrt 18

7. Zaključak 19

Poveznica na Overleaf projekt 20

Popis literature 22

Popis slika 23

1. Uvod

U posljednjem desetljeću područje umjetne inteligencije, a specifično dubokog učenja, doživjelo je eksponencijalni rast, transformirajući način na koji računala interpretiraju vizualni svijet. U središtu ove revolucije nalaze se konvolucijske neuronske mreže, klasa dubokih neuronskih mreža koje su se pokazale superiornima u zadaćama klasifikacije slika, detekcije objekata i segmentacije. Od ranih arhitektura poput LeNet, koje su postavile temelje prepoznavanja uzoraka inspiriranih biološkim procesima, do današnjih sofisticiranih modela, fokus istraživanja postupno se pomicao. Dok je inicijalni cilj bio isključivo povećanje točnosti modela kroz dodavanje sve većeg broja slojeva i parametara, suvremeni izazovi usmjereni su prema optimizaciji i učinkovitosti. Danas, s porastom primjene umjetne inteligencije na mobilnim uređajima i u sustavima s ograničenim resursima, vodeća primjena postaje razvoj arhitektura koje zadržavaju visoku preciznost uz drastično smanjenje računalne zahtjevnosti.

Ovaj rad bavi se analizom i usporedbom različitih pristupa klasifikaciji slika na standardiziranom skupu podataka CIFAR-10. Cilj rada nije samo implementirati modele, već kritički sagledati evoluciju CNN arhitektura, od njenih osnovnih modela izrađenih "od nule" do primjene naprednih tehnika prijenosa učenja. U teorijskom dijelu rada oslanjamo se na temeljne koncepte definirane u literaturi, posebice na inovacije koje su uvele tvrtke poput Googlea. Konkretno istražujemo arhitekturu MobileNet, koja je redefinirala učinkovitost uvođenjem dubinski razdvojenih konvolucija. Ova tehnika omogućuje značajno smanjenje broja parametara razdvajanjem procesa filtriranja i kombiniranja značajki, čime mreže postaju "lakše" i brže. S druge strane, analiziramo arhitekturu EfficientNet, koja uvodi koncept složenog skaliranja. Za razliku od tradicionalnih pristupa, EfficientNet simultano optimizira dubinu, širinu i rezoluciju mreže.

Praktični dio rada temelji se programskom jeziku Python i biblioteci TensorFlow Keras. Kroz rad ispitano je kako različiti faktori utječu na konačnu točnost klasifikacije i stabilnost procesa učenja.

2. Definicije neuronskih mreža

Duboko učenje je grana strojnog učenja koja omogućuje računalima da uče slojevite reprezentacije podataka. Umjesto da se uči samo jedna ili dvije značajke, deep learning modeli u obliku neuronskih mreža uče niz slojeva značajki. Onoliko koliko slojeva se dodijeli modelu zapravo označava tu dubinu modela. U dubokom učenju upravo ti slojevi uče se pomoću neuronskih mreža koje su strukturirane u doslovnim slojevima naslaganim jedan na drugi. Pri tome se svaki sloj obrađuje i pročišćuje informacije iz prethodnog sloja. To omogućuje modelima da automatski prepoznaju složene obrasce, poput objekata na slikama. Modeli dubokog učenja često koriste stotine slojeva, što im omogućuje učenje vrlo složenih i apstraktnih reprezentacija podataka. Termin neuronska mreža inspirirana je ljudskim mozgom i pokušajem razumijevanja kako mozak funkcionira, no nema stvarne povezanosti između rada modela dubokog učenja i mozga.[1]

Postoje različiti modeli dubokog učenja. Svi ti modeli dijele istu osnovnu ideju, a najuže bi ih se moglo razdijeliti s obzirom na svrhu. Postoje modeli za klasifikaciju slika, obradu teksta i jezika, obradu zvuka,... U svrhu ovog rada obraditi će se samo izabrani modeli (arhitekture) za klasifikaciju slika.

3. Arhitekture neuronske mreže

U ovom poglavlju teoretski će se obraditi izabrane arhitekture konvolucijskih neuronskih mreža. Poglavlje započinjemo kratkim povijesnim pregledom LeNet arhitekture, koja je postavila temelje modernih konvolucijskih neuronskih mreža. Zatim se fokusiramo na MobileNet i EfficientNet arhitekture gdje će se istaknuti značajke o svakoj arhitekturi i njihovim verzijama. Ove dvije arhitekture odabrane su zbog razlike u njihovim, može se reći "filozofijama" i kasnije su obrađene na primjeru.

3.1. LeNet

Iako se u radu neće implementirati i daljnje objašnjavati smatramo da je važno spomenuti arhitekturu LeNet. LeNet je razvijena u devedesetima od strane od strane Yann LeCuna i suradnika. To je jedna od prvih konvolucijskih neuronskih mreža i temelj dubokog učenja kakav je danas poznat. U tadašnje vrijeme, strojno učenje ovisilo je jako o čovjeku i njegovim odlukama koji elementi su važni. Ako čovjek nije dobro odabrao relevante elemente, model nije radio. Upravo zbog toga došlo je do ideje da se model može sam naučiti izdvajati relevantne značajke iz sirovih podataka slike, kroz proces treniranja i optimizacije. [2]

Ono što je ovaj model izdvajalo od postojećih neuronskih mreža bila je uporaba konvolucijskih slojeva: slojeva koji primjenjuju filtere na male dijelove ulazne slike, što mreži omogućuje otkrivanje uzoraka poput rubova. Ti filteri bili su dijeljeni po različitim dijelovima slike, što je drastično smanjilo broj parametara i omogućilo modelu bolje generaliziranje. Nakon konvolucijskih slojeva, mreža je primjenjivala subsampling ili pooling kako bi smanjila prostorne dimenzije te se na taj način omogućilo učinkovitije prepoznavanje. Kroz proces treniranja mreža uči kako prepoznati lokalne uzorke poput rubova i zakrivljenosti te klasificirati znamenke točno i pouzdano. Iako jednostavan po današnjim standardima, LeNet-5 je uveo koncept end-to-end učenja, integrirajući ekstrakciju značajki i klasifikaciju u jedan sustav kojeg se može trenirati, što je postalo temelj za razvoj svih modernih CNN arhitektura.[2]

3.2. MobileNet

MobileNet je arhitektura konvolucijskih neuronskih mreža namjenjena za mobilne i male uređaje. [3] Ova arhitektura razvijena je od strane tvrtke Google.[4]MobileNet koristi dubinske (depthwise) konvolucije koje značajno smanjuju broj parametara u odnosu na druge neuronske mreže, čime nastaje lagana i učinkovita duboka neuronska mreža. Dubinski razdvojena konvolucija temelji se na ideji da se dubina i prostorna dimenzija filtra mogu odvojiti. Ova konvolucija sastoji se od dviju operacija filtriranje svakog ulaznog kanala (depthwise convolution) i kombinacija informacija iz svih kanala te stvaranje novih značajki (pointwise convolution).[5]

MobileNet v1 je prva verzija MobileNet arhitekture, predstavljena 2017. godine, s ciljem razvoja brze i računski učinkovite konvolucijske neuronske mreže. Već sljedeće godine, Google predstavlja MobilenetV2 nadogradnju v1 arhitekture te je oko 0,3 puta manji i brži, uz približno

1% veću točnost u odnosu na MobileNet v1, zahvaljujući invertiranim rezidualnim blokovima i linearnim bottleneck slojevima. MobileNetV3 (2019) dodatno optimizira mrežu te je u usporedbi s MobileNetom v2 oko 2 puta brži i 30% manji, ali uz oko 3% manju točnost, što predstavlja kompromis između performansi i učinkovitosti.[5]

3.3. EfficientNet

EfficientNet predstavlja značajan iskorak u načinu na koji pristupamo dizajniranju i optimizaciji konvolucijskih neuronskih mreža. Za razliku od ranijih pristupa koji su se fokusirali na nasumično povećanje dubine, širine ili rezolucije slike kako bi se poboljšala točnost modela, EfficientNet uvodi princip složenog skaliranja (compound scaling). Tu arhitekturu su 2019. godine predstavili istraživači iz Google Brain tima, Mingxing Tan i Quoc V. Le. Arhitektura temelji na hipotezi da uravnoteženo skaliranje svih triju dimenzija mreže istovremeno dovodi do znatno boljih performansi uz manju potrošnju računalnih resursa.

Analizirajući strukturu bitno je primijetiti da se EfficientNet ne oslanja na ručno dizajniranje slojeva, već koristi tehniku pretraživanja arhitekture kako bi definirao osnovnu mrežu, nazvanu EfficientNet-B0; ta bazna mreža optimizirana je za postizanje visoke točnosti uz minimalne operacije s pomičnim zarezom. Gradivni blokovi EfficientNeta temelje se na MBConv slojevima koji su slični onima koje smo vidjeli kod MobileNetV2 arhitekture, ali s dodatkom mehanizma "Squeeze-and-Excitation" koji mreži omogućuje da adaptivno rekalibrira važnost značajki po kanalima.

Umjesto proizvoljnog dodavanja slojeva, EfficientNet koristi složeni koeficijentϕ(fi) koji uniformno skalira širinu, dubinu i rezoluciju mreže. Na primjer, ako imamo dvostruko više računalnih resursa, formulom se točno određuje koliko treba povećati svaku od dimenzija kako bi se zadržala optimalna učinkovitost. Kroz seriju modela od B0 do B7, pokazalo se da ova metoda nadmašuje tadašnje modele. EfficientNet-B7 postigao je 84.3% top-1 točnosti na ImageNet skupu podataka, budući 8.4 puta manji i 6.1 puta brži pri inferenciji od tada vodećeg GPipe modela.

EfficientNet arhitektura ukazuje na to da budućnost dubokog učenja ne leži nužno u stvaranju sve većih i masivnijih modela, već u pametnijem dizajnu koji razumije međuovisnost dimenzija mreže.

4. Opis implementacije

Kao temelj za implementaciju obiju analiziranih arhitektura korišten je programski kod s laboratorijskih vježbi, kojeg smo potom prilagodile i nadogradile specifičnostima svakog pojedinog modela.

U sklopu eksperimentalnog dijela, fokusirale smo se na manipulaciju ključnim parametrima kako bismo ispitali njihov utjecaj na ponašanje mreža. Tako je kod MobileNet arhitekture promjenom ulazne rezolucije sa 32 na 96 piksela izravno demonstrirano kako skaliranje prostornih dimenzija utječe na performanse i strukturu mreže. S druge strane, analiza EfficientNet arhitekture usmjerena je na ispitivanje odnosa normalizacije i njezinog utjecaja na model, pri čemu smo usporedile učinkovitost automatske preprocesne normalizacije unutar slojeva mreže naspram ručne normalizacije podataka dijeljenjem s 255.

4.1. Rješenje sa laboratorijskih vježbi

Na laboratorijskim vježbama korišten je CIFAR-10 je skup podataka od 60 000 slika u boji veličine 32x32px. U tom skupu nalazi se 60 000 slika koje se dijele u 10 kategorija: Zrakoplov, automobil, ptica, mačka, jelen, pas, žaba, konj, brod i kamion. Ovaj skup podataka je podijeljen na skup za treniranje u kojem se nalazi 50 000 slika i skup za testiranje u kojem je 10 000 slika. CIFAR-10 skup podataka biti će korišten i u analiziranim arhitekturama u nastavku ovog rada.

```
model = Sequential()

# CONVOLUTIONAL LAYER
model.add(Conv2D(filters=32, kernel_size=(3,3),input_shape=(28, 28, 1), activation='
 relu',))
# POOLING LAYER
model.add(MaxPool2D(pool_size=(2, 2)))

# FLATTEN IMAGES FROM 28 by 28 to 784 BEFORE FINAL LAYER
model.add(Flatten())

# 128 NEURONS IN DENSE HIDDEN LAYER
model.add(Dense(units=128, activation='relu'))

# LAST LAYER IS THE CLASSIFIER, THUS 10 POSSIBLE CLASSES
model.add(Dense(units=10, activation='softmax'))
```

Priloženi kod je prikaz CNN-a (Concoutional Neural Network, hr: konvolucijske neuronske mreže) koji je namijenjen za klasifikaciju slika po navedenim klasama. Prikazani model radi tako da uspoređuje slike, dio po dio.[6]Na ulazu konvolucijske mreže može biti monokromatska slika ili slika u boji.[7] Razlika između kasnije testiranih arhitektura i ove je u tome da MobileNet i EfficentNet na ulazu traže boje pa se monokromatskim slikama moraju dodavati tri kanala.

Ovaj model sadrži 5 slojeva. Prvi sloj je Conv2D sloj[8] koji sadržava 32 filtera (ozna-

čava dimenziju izlaznog prostora), kernel_size koji označava veličinu konvolucijskog prozora, input_shape koja označava dimenzije ulaznih podatka i funkciju aktivacije koja je primjenjena. Sljedeći sloj je model.add add(MaxPool2D(pool_size=(2, 2))) koji sažima podatke tako da izdvaja maksimalnu vrijednost iz svakog kanala. Pomoću pool_size=(2, 2) određena je veličina prozora za sažimanje.[9] Treći sloj Flatten služi za izravnavanje ulaznih podataka u linearni niz bez mijenjanja broja uzoraka u seriji.[10] Četvrti i peti sloj su Dense slojevi. [11]Units označava dimenziju izlaznog prostora ovog sloja. U kodu sa laboratorijskih vježbi, u četvrtom sloju ta dimenzija iznosi 128, što znači da se u ovom sloju ulazni podatci pretvaraju u 128 informacija o slici. U zadnjem sloju kod aktivacije koristi se funkcija softmax koja rezultate pretvara u vjerojatnosti, čime se omogućuje konačno razvrstavanje podataka u jednu od 10 zadanih kategorija sa početka.[11]

4.2. MobileNetV2

Kao što je već bilo navedeno, koristiti će se CIFAR-10 skup podataka za provedbu analize. Kao bazni model korišten je MobileNetV2 iz TensorFlow Keras biblioteke pretreniran na ImageNet-u.

```
(x_treniranje, y_treniranje), (x_testiranje, y_testiranje) = cifar10.load_data()
```

Skup slika CIFAR-10 je učitan pri čemu su slike i pripadne oznake podijeljene na trening skup (x_treniranje, y_treniranje) i test skup (x_testiranje, y_testiranje).

```
print("X_treniranje izvorni oblik", x_treniranje.shape)
print("y_train izvorni oblik", y_treniranje.shape)
print("X_test izvorni oblik", x_testiranje.shape)
print("y_test izvorni oblik", y_testiranje.shape)
```

Dimenzije trening i test podataka su provjerene pomoću shape atributa, čime se osigurava da su slike i pripadne oznake u očekivanom obliku prije daljnje obrade.

```
y_kat_treniranje = to_categorical(y_treniranje,10)
y_kat_testiranje = to_categorical(y_testiranje,10)
```

Oznake klasa iz CIFAR-10 su inicijalno predstavljene kao brojevi od 0 do 9, pretvorene su u one-hot kodiranje pomoću funkcije to_categorical. Time se svaka klasa prikazuje kao vektor duljine 10 s jedinicom na poziciji odgovarajuće klase i nulama na svim ostalim pozicijama.

```
print("y_train izvorni oblik", y_kat_treniranje.shape)
print("y_test izvorni oblik", y_kat_testiranje.shape)
```

Nakon pretvorbe oznaka u one-hot format, provjerene su dimenzije dobivenih nizova (y_kat_treniranje i y_kat_testiranje) kako bi se osiguralo da mreža prima podatke u očekivanom obliku za treniranje i evaluaciju.

```
X_treniranje_norm = x_treniranje/255
X_testiranje_norm = x_testiranje/255
```

Pikseli slika su normalizirani dijeljenjem s 255 kako bi vrijednosti bile u rasponu 0–1, što omogućuje stabilnije i učinkovitije treniranje neuronske mreže.

```
bazni_model = MobileNetV2(
    input_shape=(32,32,3),
    include_top=False,
    weights='imagenet'
)
```

Kao bazni model korišten je MobileNetV2 pretreniran na ImageNet-u, s prilagođenim ulaznim oblikom (32×32×3) i bez vršnog klasifikacijskog sloja, kako bi ga mogli nadograditi sa željenim slojevima.

```
bazni_model.trainable = False
```

Bazni MobileNetV2 model je zamrznut (trainable = False), što znači da se njegove pretrenirane težine ne ažuriraju tijekom treniranja, a mreža uči samo dodane slojeve.

```
x = bazni_model.output
```

Varijabla x prima izlaz iz baznog MobileNetV2 modela, što predstavlja ekstraktirane značajke iz slika i služi kao polazna točka za dodavanje novih slojeva.

```
x = GlobalAveragePooling2D()(x)
```

Prvi sloj. Na izlaz baznog modela primijenjen je GlobalAveragePooling2D sloj koji sažima prostorne značajke u vektor, zadržavajući najvažnije informacije i smanjujući dimenzije podataka za sljedeće slojeve.

```
x = Dense(128, activation='relu')(x)
```

Drugi sloj. Svaki od 128 neurona prima sve ulaze iz prethodnog sloja i relu aktivacijom omogućuje modelu da uči apstraktne značajke specifične za CIFAR-10.

```
output = Dense(10, activation='softmax')(x)
```

Treci sloj. Izlazni sloj s 10 neurona i softmax aktivacijom pretvara naučene značajke u vjerojatnosti pripadnosti svakoj od 10 klasa.

```
model = Model(inputs=bazni_model.input, outputs=output)
```

Konačni model je definiran spajanjem ulaza baznog MobileNetV2 modela s navedenim slojevima.

```
model.compile(loss='categorical_crossentropy',
              optimizer='adam',
              metrics=['accuracy'])
```

Kompilacija modela odvija se s navedenim loss, optimizer i metrics parametrima. Za funkciju gubitka koristi se 'categorical_crossentropy', za optimizaciju koristi se algoritam adam i prati se točnost modela tijekom njegovog treninga i evaluacije.

```
early_stop = EarlyStopping(monitor='val_loss',patience=3)
model.fit(X_treniranje_norm, y_kat_treniranje,
          epochs=10,
          #batch_size=,
          validation_data=(X_testiranje_norm, y_kat_testiranje),
          callbacks=[early_stop])
```

Koristi se rano zaustavljanje ako model pokazuje prenaučenost. Treniranje modela odvija se na 10 epoha što znači da model prolazi kroz cijeli skup za treniranje 10 puta. Batch_size će se kasnije objasniti na primjeru.

```
print(model.metrics_names)
print(model.evaluate(X_testiranje_norm,y_kat_testiranje,verbose=0))
```

U ovom dijelu se model evaluira odnosno vraćaju se vrijednosti funkcije gubitka i definiranih metrika (accuracy) na test skupu.

```
predvidanja = model.predict(X_testiranje_norm)
predvidi_klase = np.argmax(predvidanja, axis=1)
print(classification_report(y_testiranje,predvidi_klase))
```

Provjera točnosti modela i koliko dobro model predviđa klase.

4.3. EfficientNetB1

Jednako kao i u prethodnom slučaju, analiza se provodi na CIFAR-10 skupu podataka, no ovoga puta kao temeljni model koristimo EfficientNetB1, koji zahtijeva specifičnu pripremu podataka. Učitavanje CIFAR-10 skupa podataka provedeno je na standardan način, odvajajući podatke na skup za treniranje i skup za testiranje.

```
(x_treniranje, y_treniranje), (x_testiranje, y_testiranje) = cifar10.load_data()
```

Inicijalna provjera dimenzija potvrđuje da su podaci ispravno učitani u formatu (broj_slika, 32, 32, 3), što odgovara izvornoj rezoluciji CIFAR-10 skupa.

```
print("X_treniranje izvorni oblik", x_treniranje.shape)
print("y_train izvorni oblik", y_treniranje.shape)
print("X_test izvorni oblik", x_testiranje.shape)
print("y_test izvorni oblik", y_testiranje.shape)
```

Za razliku od MobileNetV2 pristupa, ovdje je napravljena promjena rezolucije. Izvorne slike dimenzija 32×32 piksela povećavaju se na 96×96 piksela. Ovo je ključan korak za EfficientNetB1 arhitekturu koja je optimizirana za rad s većim rezolucijama, čime se omogućuje bolje prepoznavanje finijih značajki koje bi se izgubile na nižoj rezoluciji.

```
x_treniranje_96 = tf.image.resize(x_treniranje, (96, 96))
x_testiranje_96 = tf.image.resize(x_testiranje, (96, 96))
```

Oznake klasa se pretvaraju u *one-hot* kodirani zapis pomoću funkcije `to_categorical`, čime se pripremaju za izlazni sloj mreže koji koristi `softmax` aktivaciju. Provjerom dimenzija osiguravamo ispravnost transformacije.

```
y_kat_treniranje = to_categorical(y_treniranje, 10)
y_kat_testiranje = to_categorical(y_testiranje, 10)

print("y_train izvorni oblik", y_kat_treniranje.shape)
print("y_test izvorni oblik", y_kat_testiranje.shape)
```

Funkcija `preprocess_input` iz modula `efficientnet` automatski skalira i normalizira ulazne podatke. Time osigurava maksimalnu kompatibilnost s pretreniranim težinama. Podaci se prethodno prebacuju u `float32` format.

```
X_treniranje_norm = preprocess_input(tf.cast(x_treniranje_96, tf.float32))
X_testiranje_norm = preprocess_input(tf.cast(x_testiranje_96, tf.float32))
```

Kao temelj mreže inicijalizira se EfficientNetB1 model sa značajkama naučenim na ImageNet-u. Definiran je novi ulazni oblik ($96 \times 96 \times 3$) koji odgovara prethodno skaliranim slikama, a parametar `include_top=False` isključuje zadnji klasifikacijski sloj kako bismo mogli dodati vlastiti.

```
bazni_model = EfficientNetB1(
    input_shape=(96, 96, 3),
    include_top=False,
    weights='imagenet'
)
```

Zamrzavanjem baznog modela sprječavamo uništavanje naučenih značajki tijekom inicijalne faze treniranja, fokusirajući proces učenja isključivo na nove slojeve koje dodajemo.

```
bazni_model.trainable = False
```

Na izlaz baznog modela dodana je vlastita arhitektura glave (`GlobalAveragePooling2D` za smanjenje dimenzionalnosti, `Dense` sloj sa 128 neurona i `ReLU` aktivacijom za učenje specifičnih uzoraka, te konačni `Dense` izlazni sloj s 10 neurona i `softmax` aktivacijom za klasifikaciju u 10 kategorija).

```
x = bazni_model.output
x = GlobalAveragePooling2D()(x)
x = Dense(128, activation='relu')(x)
output = Dense(10, activation='softmax')(x)
```

Konačni model je sastavljen povezivanjem ulaza i novog izlaza. Kompajliranje se vrši uz `adam` optimizator i `categorical_crossentropy` funkciju gubitka.

```
model = Model(inputs=bazni_model.input, outputs=output)

model.compile(
    loss='categorical_crossentropy',
    optimizer='adam',
    metrics=['accuracy']
)
```

Prikaz sažetka modela omogućuje uvid u broj parametara. Treniranje se provodi kroz 10 epoha uz korištenje `EarlyStopping` mehanizma koji prati `val_loss` i zaustavlja proces ako se pogreška na validacijskom skupu ne smanji kroz 3 uzastopne epohe, čime se sprječava prenaučenost.

```
model.summary()
early_stop = EarlyStopping(monitor='val_loss', patience=3)
model.fit(
    X_treniranje_norm, y_kat_treniranje,
```

```
    epochs=10,
    #batch_size=128,
    validation_data=(X_testiranje_norm, y_kat_testiranje),
    callbacks=[early_stop]
)
```

Evaluacija modela na testnom skupu podataka vraća konačnu točnost i vrijednost funkcije gubitka.

```
print(model.metrics_names)
print(model.evaluate(X_testiranje_norm, y_kat_testiranje, verbose=0))
```

Generiranjem predikcija na testnom skupu i usporedbom sa stvarnim oznakama kreira se klasifikacijski izvještaj. On pruža detaljan uvid u performanse modela po svakoj klasi kroz metrike *precision*, *recall* i F1-mjere.

```
predvidanja = model.predict(X_testiranje_norm)
predvidi_klase = np.argmax(predvidanja, axis=1)

print(classification_report(y_testiranje, predvidi_klase))
```

5. Prikaz rada

5.1. Kod sa laboratorijskih vježbi

Na slici ispod može se vidjeti vrijeme i preciznost koda sa laboratorijskih vježbu po epohama. Ukupno trajanje iznosi 582 sekunde.

```
Epoch 1/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 65s 41ms/step - accuracy: 0.3650 - loss: 1.7450 - val_accuracy: 0.4600 - val_loss: 1.6817
Epoch 2/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 80s 39ms/step - accuracy: 0.5801 - loss: 1.2007 - val_accuracy: 0.5839 - val_loss: 1.1902
Epoch 3/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 81s 39ms/step - accuracy: 0.6482 - loss: 1.0077 - val_accuracy: 0.6419 - val_loss: 1.0344
Epoch 4/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 82s 38ms/step - accuracy: 0.6936 - loss: 0.8813 - val_accuracy: 0.4724 - val_loss: 1.6320
Epoch 5/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 82s 39ms/step - accuracy: 0.7260 - loss: 0.7950 - val_accuracy: 0.6711 - val_loss: 0.9764
Epoch 6/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 82s 39ms/step - accuracy: 0.7555 - loss: 0.7141 - val_accuracy: 0.6185 - val_loss: 1.1669
Epoch 7/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 59s 38ms/step - accuracy: 0.7751 - loss: 0.6590 - val_accuracy: 0.6852 - val_loss: 0.9646
Epoch 8/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 83s 39ms/step - accuracy: 0.7947 - loss: 0.6031 - val_accuracy: 0.6717 - val_loss: 1.1065
Epoch 9/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 61s 39ms/step - accuracy: 0.8101 - loss: 0.5495 - val_accuracy: 0.6790 - val_loss: 1.1286
Epoch 10/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 84s 40ms/step - accuracy: 0.8235 - loss: 0.5203 - val_accuracy: 0.6851 - val_loss: 1.0271
<keras.src.callbacks.history.History at 0x7f3edae9fe90>
```

Slika 1: Prikaz točnosti po epohama koda sa laboratorijskih vježbi

```
print(model.metrics_names)
print(model.evaluate(X_test_norm,y_cat_test,verbose=0))

['loss', 'accuracy']
[1.2075093984603882, 0.6789000034332275]
```

Slika 2: Točnost modela sa laboratorijskih vježbi

Točnost modela je 68%.

	precision	recall	f1-score	support
0	0.74	0.69	0.71	1000
1	0.87	0.78	0.82	1000
2	0.59	0.58	0.58	1000
3	0.48	0.54	0.51	1000
4	0.54	0.71	0.61	1000
5	0.63	0.47	0.54	1000
6	0.81	0.71	0.76	1000
7	0.68	0.77	0.72	1000
8	0.80	0.78	0.79	1000
9	0.74	0.77	0.76	1000
accuracy			0.68	10000
macro avg	0.69	0.68	0.68	10000
weighted avg	0.69	0.68	0.68	10000

Slika 3: Analiza modela sa laboratorijskih vježbi po klasama

Tu se može vidjeti da najmanju točnost model ima kod klase 3 odnosno da najteže prepoznaje mačke od svih ostalih klasa.

5.2. MobileNetV2

Kod pokretanja arhiteture MobilenetV2 s istim parametrima kao kod s laboratorijskih vježbi dobije se sljedeće:

```
Epoch 1/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 63s 36ms/step - accuracy: 0.2943 - loss: 1.9418 - val_accuracy: 0.3238 - val_loss: 1.8682
Epoch 2/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 53s 34ms/step - accuracy: 0.3310 - loss: 1.8419 - val_accuracy: 0.3300 - val_loss: 1.8404
Epoch 3/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 54s 35ms/step - accuracy: 0.3449 - loss: 1.8040 - val_accuracy: 0.3382 - val_loss: 1.8156
Epoch 4/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 54s 35ms/step - accuracy: 0.3532 - loss: 1.7768 - val_accuracy: 0.3392 - val_loss: 1.8126
Epoch 5/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 55s 35ms/step - accuracy: 0.3614 - loss: 1.7551 - val_accuracy: 0.3464 - val_loss: 1.8007
Epoch 6/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 54s 35ms/step - accuracy: 0.3667 - loss: 1.7369 - val_accuracy: 0.3438 - val_loss: 1.8016
Epoch 7/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 55s 35ms/step - accuracy: 0.3752 - loss: 1.7205 - val_accuracy: 0.3502 - val_loss: 1.8004
Epoch 8/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 54s 35ms/step - accuracy: 0.3771 - loss: 1.7063 - val_accuracy: 0.3530 - val_loss: 1.7966
Epoch 9/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 55s 35ms/step - accuracy: 0.3832 - loss: 1.6935 - val_accuracy: 0.3528 - val_loss: 1.7924
Epoch 10/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 55s 35ms/step - accuracy: 0.3865 - loss: 1.6816 - val_accuracy: 0.3534 - val_loss: 1.7933
['loss', 'compile_metrics']
[1.793286681175232, 0.35339999198913574]
```

Slika 4: Rezultati treniranja MobilenetV2 arhitekture po epohama

Ukupno vrijeme trajanja iznosi 552 sekunde, a točnost iznosi samo 35%.

	precision	recall	f1-score	support
0	0.41	0.39	0.40	1000
1	0.28	0.41	0.33	1000
2	0.28	0.20	0.23	1000
3	0.32	0.24	0.28	1000
4	0.37	0.42	0.39	1000
5	0.32	0.23	0.27	1000
6	0.43	0.46	0.44	1000
7	0.37	0.37	0.37	1000
8	0.33	0.49	0.39	1000
9	0.43	0.34	0.37	1000
accuracy			0.35	10000
macro avg	0.35	0.35	0.35	10000
weighted avg	0.35	0.35	0.35	10000

Slika 5: Analiza modela s MobileNetV2 arhitekturom po klasama

Razlog tome je što MobilenetV2 radi s ulaznim slikama koje imaju najmanje 94*94px.Kako bi popravili ovako malenu točnost nakon uvoza CIFAR10 biblioteke dodajemo sljedeće dvije li-

nije:

```
x_treniranje = tf.image.resize(x_treniranje,[96,96])
x_testiranje = tf.image.resize(x_testiranje,[96,96])
```

One služe kako bi se slike iz CIFAR-a povećale sa 32*32px na 96*96px.[12] Zbog toga će se vrijeme trajanja po epohi produljiti no nakon toga dobivamo sljedeće:

```
Epoch 1/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 192s 117ms/step - accuracy: 0.7620 - loss: 0.6904 - val_accuracy: 0.7912 - val_loss: 0.5984
Epoch 2/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 180s 115ms/step - accuracy: 0.8141 - loss: 0.5295 - val_accuracy: 0.8002 - val_loss: 0.5902
Epoch 3/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 183s 117ms/step - accuracy: 0.8347 - loss: 0.4685 - val_accuracy: 0.8052 - val_loss: 0.5734
Epoch 4/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 174s 111ms/step - accuracy: 0.8492 - loss: 0.4256 - val_accuracy: 0.7900 - val_loss: 0.6129
Epoch 5/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 174s 111ms/step - accuracy: 0.8634 - loss: 0.3816 - val_accuracy: 0.8095 - val_loss: 0.5706
Epoch 6/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 180s 115ms/step - accuracy: 0.8750 - loss: 0.3468 - val_accuracy: 0.7979 - val_loss: 0.6405
Epoch 7/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 164s 105ms/step - accuracy: 0.8889 - loss: 0.3105 - val_accuracy: 0.8017 - val_loss: 0.6598
Epoch 8/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 166s 106ms/step - accuracy: 0.8994 - loss: 0.2797 - val_accuracy: 0.8052 - val_loss: 0.6338
['loss', 'compile_metrics']
[0.6337736248970032, 0.8051999807357788]
```

Slika 6: Prikaz epoha nakon povećanja slika za arhitekturu MobileNetV2

	precision	recall	f1-score	support
0	0.85	0.83	0.84	1000
1	0.92	0.89	0.90	1000
2	0.78	0.73	0.76	1000
3	0.63	0.71	0.67	1000
4	0.69	0.83	0.76	1000
5	0.81	0.63	0.71	1000
6	0.80	0.86	0.83	1000
7	0.84	0.81	0.83	1000
8	0.87	0.90	0.89	1000
9	0.90	0.87	0.88	1000
accuracy			0.81	10000
macro avg	0.81	0.81	0.81	10000
weighted avg	0.81	0.81	0.81	10000

Slika 7: Analiza modela s MobileNetV2 arhitekturom po klasama nakon povećanja slika

Možemo vidjeti da je točnost porasla na 81% što je jako velika razlika i jako dobar rezultat. Najmanja točnost je i dalje ostala kod klasa sa mačkama.

5.3. EfficientNetB1

Kod pokretanja arhitekture EfficientNetB1, uz primjenu `preprocess_input` funkcije i prethodno povećanje rezolucije slika na 96×96 piksela, dobiveni su sljedeći rezultati treniranja:

Model doseže točnost od 81%. Ukupno vrijeme trajanja iznosi 1413 sekunde. Nakon 8. etape došlo je do ranog zaustavljanja radi prenaučenosti modela.

Nakon povećanja veličine slika i ovom modelu je najveći problem klasa sa slikama mački.

```
X_treniranje_norm = preprocess_input(tf.cast(x_treniranje_96, tf.float32))
X_testiranje_norm = preprocess_input(tf.cast(x_testiranje_96, tf.float32))
```

Slika 8: Primjena preprocess_input funkcije

```
 Total params: 6,740,497 (25.71 MB)
 Total params: 6,740,497 (25.71 MB)
 Trainable params: 165,258 (645.54 KB)
 Non-trainable params: 6,575,239 (25.08 MB)
 Trainable params: 165,258 (645.54 KB)
 Non-trainable params: 6,575,239 (25.08 MB)
Epoch 1/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 167s 99ms/step - accuracy: 0.8084 - loss: 0.5642 - val_accuracy: 0.8422 - val_loss: 0.4601
Epoch 2/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 143s 91ms/step - accuracy: 0.8535 - loss: 0.4224 - val_accuracy: 0.8455 - val_loss: 0.4441
Epoch 3/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 146s 93ms/step - accuracy: 0.8723 - loss: 0.3623 - val_accuracy: 0.8447 - val_loss: 0.4510
Epoch 4/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 151s 97ms/step - accuracy: 0.8886 - loss: 0.3133 - val_accuracy: 0.8485 - val_loss: 0.4590
Epoch 5/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 150s 96ms/step - accuracy: 0.9025 - loss: 0.2778 - val_accuracy: 0.8523 - val_loss: 0.4569
['loss', 'compile_metrics']
[0.4569475054740906, 0.8522999882698059]
```

Slika 9: Rezultati treniranja po epohama (EfficientNetB1)

Model doseže visoku točnost od 85.22%. Ukupno vrijeme trajanja za 5 epoha iznosi 757 sekundi. Nakon 5. epohe došlo je do ranog zaustavljanja jer model više nije značajno smanjivao funkciju gubitka, čime je spriječena prenaučenost.

```
              precision    recall  f1-score   support

           0       0.89      0.84      0.86      1000
              precision    recall  f1-score   support

           0       0.89      0.84      0.86      1000

           0       0.89      0.84      0.86      1000
           1       0.92      0.89      0.91      1000
           0       0.89      0.84      0.86      1000
           1       0.92      0.89      0.91      1000
           1       0.92      0.89      0.91      1000
           2       0.86      0.82      0.84      1000
           2       0.86      0.82      0.84      1000
           3       0.72      0.74      0.73      1000
           4       0.77      0.87      0.82      1000
           4       0.77      0.87      0.82      1000
           5       0.81      0.78      0.80      1000
           5       0.81      0.78      0.80      1000
           6       0.87      0.89      0.88      1000
           7       0.89      0.87      0.88      1000
           8       0.91      0.90      0.91      1000
           9       0.88      0.91      0.90      1000

    accuracy                           0.85     10000
   macro avg       0.85      0.85      0.85     10000
weighted avg       0.85      0.85      0.85     10000
```

Slika 10: Klasifikacijski izvještaj uspješnog modela

Ovdje možemo vidjeti da model postiže konzistentne rezultate kroz većinu klasa. Slično kao i kod MobileNet arhitekture, najmanju točnost (f1-mjera 0.73) model ima kod klase 3, odnosno kod prepoznavanja mačaka. Ostale klase poput automobila (klasa 1) prepoznate su s vrlo visokom točnošću od 91%. [13]

Nakon toga, provedena je ručna normalizacija. Umjesto korištenja `preprocess_input` funkcije iz biblioteke EfficientNet, slike su normalizirane dijeljenjem s 255 sa čime je svedena vrijednost piksela na raspon 0-1. Rezolucija je zadržana na 96×96. Cilj je bio prikazati osjetljivost naprednih arhitektura na distribuciju ulaznih podataka.

```
X_treniranje_norm = x_treniranje / 255
X_testiranje_norm = x_testiranje / 255
```

Slika 11: Kod s ručnom normalizacijom (dijeljenje s 255)

```
Total params: 6,740,497 (25.71 MB)
Trainable params: 165,258 (645.54 KB)
Non-trainable params: 6,575,239 (25.08 MB)
Epoch 1/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 140s 86ms/step - accuracy: 0.0984 - loss: 2.3044 - val_accuracy: 0.1000 - val_loss: 2.3027
Epoch 2/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 148s 94ms/step - accuracy: 0.0979 - loss: 2.3028 - val_accuracy: 0.1000 - val_loss: 2.3027
Epoch 3/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 167s 107ms/step - accuracy: 0.0980 - loss: 2.3029 - val_accuracy: 0.1000 - val_loss: 2.3026
Epoch 4/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 178s 114ms/step - accuracy: 0.0984 - loss: 2.3028 - val_accuracy: 0.1000 - val_loss: 2.3027
Epoch 5/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 179s 114ms/step - accuracy: 0.0986 - loss: 2.3027 - val_accuracy: 0.1000 - val_loss: 2.3027
Epoch 6/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 171s 109ms/step - accuracy: 0.0970 - loss: 2.3028 - val_accuracy: 0.1000 - val_loss: 2.3026
['loss', 'compile_metrics']
[2.3026351928710938, 0.10000000149011612]
```

Slika 12: Rezultati neuspješnog treniranja

Točnost modela iznosi samo 10%, a funkcija gubitka stagnira na vrijednosti oko 2.30 kroz sve epohe. Model nije uspio naučiti nikakve značajke.

	precision	recall	f1-score	support
	precision	recall	f1-score	support
0	0.00	0.00	0.00	1000
1	0.00	0.00	0.00	1000
0	0.00	0.00	0.00	1000
1	0.00	0.00	0.00	1000
2	0.00	0.00	0.00	1000
1	0.00	0.00	0.00	1000
2	0.00	0.00	0.00	1000
2	0.00	0.00	0.00	1000
3	0.00	0.00	0.00	1000
4	0.00	0.00	0.00	1000
5	0.00	0.00	0.00	1000
6	0.10	1.00	0.18	1000
7	0.00	0.00	0.00	1000
8	0.00	0.00	0.00	1000
9	0.00	0.00	0.00	1000
accuracy			0.10	10000
macro avg	0.01	0.10	0.02	10000
weighted avg	0.01	0.10	0.02	10000

Slika 13: Klasifikacijski izvještaj neuspješnog modela

Jasno je da je model potpuno podbacio. Precision, Recall i f1-mjera su 0.00 za gotovo sve klase, osim za klasu 6 gdje je model klasificirao sve uzorke, što je rezultiralo prividnim recall-om od 1.00.

Razlog ovakvog rezultata leži u specifičnosti EfficientNet arhitekture. Dok jednostavnije mreže mogu raditi s podacima u rasponu 0-1, EfficientNetB1 očekuje ulazne podatke koji su skalirani na točno određen način kako su bili i tijekom originalnog treniranja. Korištenjem funkcije `preprocess_input`, podaci se automatski prilagođavaju tim zahtjevima. Ručnim dijeljenjem s 255 narušena je distribucija podataka koju pretrenirane težine očekuju. Rezultat

toga je nemogućnost modela da izvuče korisne značajke. Ovo potvrđuje da je kod korištenja naprednih pretreniranih modela ključno koristiti pripadajuće funkcije za pretprocesiranje. [14]

6. Kritički osvrt

Kroz implementacije dobiven je detaljan uvid u ponašanje različitih arhitektura konvolucijskih neuronskih mreža na skupu podataka CIFAR-10, s posebnim naglaskom na važnost predobrade podataka i prilagodbe parametara. Kroz komparativnu analizu osnovnog modela s laboratorijskih vježbi te naprednih arhitektura MobileNetV2 i EfficientNetB1, jasno su se vidjeli izazovi i prednosti primjene prijenosa učenja.

Polazna točka bio je CNN model s laboratorijskih vježbi, koji je postigao točnost od 68%. Iako je rezultatom zadovoljavajuć ovaj model je pokazao svoja ograničenja u sposobnosti generalizacije na složenijem skupu podataka. Uvođenjem MobileNetV2 arhitekture, očekivalo se trenutačno poboljšanje, no inicijalni eksperiment rezultirao je točnošću od svega 35%. Ovaj neuspjeh bio je ključan za razumijevanje praktične izvedivosti MobileNetV2 jer je on dizajniran za slike veće rezolucije (tipično 224x224). Povećanjem ulazne rezolucije na 96x96 piksela, točnost je drastično porasla na 81%, čime je potvrđeno da je usklađivanje ulaznih dimenzija sa zahtjevima arhitekture preduvjet za uspješnu implementaciju.

Još jedan značajan uvid ovog rada proizašao je iz analize EfficientNetB1 arhitekture, koja je u optimalnim uvjetima postigla rezultat od 85.22%. Ovaj način implementacije je dokazao da uspjeh u dubokom učenju ne ovisi samo o odabiru modela, već o strogom poštivanju distribucije podataka na kojoj je model originalno treniran. Eksperiment s ručnom normalizacijom (dijeljenje s 255), koji je rezultirao potpunim kolapsom modela pokazuje to da napredne mreže nisu robusne na proizvoljne promjene u predobradi. Dok jednostavnije mreže mogu učiti iz podataka u rasponu [0, 1]. Ovaj primjer je od važan za praktičnu primjenu jer ukazuje na to da male pogreške u cjevovodu podataka mogu učiniti inače superioran model beskorisnim.

S aspekta praktične izvedivosti, oba analizirana modela pokazala su se iznimno učinkovitima. Unatoč povećanju rezolucije slika, vremena treniranja ostala su unutar prihvatljivih okvira, što potvrđuje njihovu namjenu. Ovi modeli nisu dizajnirani za masivne poslužitelje. Primjena ovakvih optimiziranih arhitektura je široka; od mobilnih aplikacija za klasifikaciju objekata u stvarnom vremenu, preko ugrađenih sustava u autonomnim vozilima, do IoT uređaja s ograničenim energetskim resursima. EfficientNetB1 se u ovom kontekstu istaknuo kao bolji izbor jer nudi bolji omjer točnosti i broja parametara u odnosu na MobileNetV2, pod uvjetom da se resursi uređaja mogu nositi s nešto složenijim matematičkim operacijama koje zahtijeva.

7. Zaključak

Provedeno istraživanje i analiza na skupu podataka CIFAR-10 pružili su dubok uvid u složenost implementacije modernih konvolucijskih neuronskih mreža. Rad je demonstrirao da odabir "state-of-the-art" modela nije garancija vrhunskih rezultata bez temeljitog razumijevanja njihovih unutarnjih mehanizama i zahtjeva za predobradom podataka. Rezultati su potvrdili da arhitektura EfficientNetB1, kada je ispravno konfigurirana, nadmašuje ostale testirane modele. Međutim, put do ovog rezultata otkrio je ključne izazove. Eksperiment s MobileNetV2 arhitekturom, koji je inicijalno rezultirao sa 35% točnosti pri originalnoj rezoluciji od 32x32 piksela je dokazao da arhitekture optimizirane za ImageNet zahtijevaju veće prostorne dimenzije kako bi njihovi duboki slojevi mogli učinkovito funkcionirati. Povećanjem rezolucije na 96x96 piksela, model je uspio iskoristiti svoj potencijal i dosegnuti 81% točnosti. Ovo potvrđuje da se "lakoća" MobileNet arhitekture odnosi na broj parametara, ali ne nužno i na fleksibilnost prema ulaznim dimenzijama. Drugi zaključak ovog rada proizlazi iz analize osjetljivosti EfficientNet modela na normalizaciju podataka. Usporedba između korištenja namjenske funkcije `preprocess_input` i ručne normalizacije dijeljenjem s 255 (što je standardna praksa za jednostavne modele) rezultirala je drastičnom razlikom; od vrhunske točnosti do potpunog kolapsa modela na 10%. MobileNet fokusira se na jednostavnu mrežu pogodnu za uređaje s ograničenim resursima koristeći smanjenja broja parametara i računanja, dok EfficientNet pokazuje princip složenog skaliranja mreže kako bi se postigla maksimalna točnost s optimalnim korištenjem resursa.

Modeli koji su trenirani na specifičnoj distribuciji očekuju identičnu distribuciju na ulazu kako bi njihove pretrenirane težine bile validne. Zanemarivanje ovog koraka čini model neupotrebljivim, bez obzira na njegovu teorijsku snagu.

U širem kontekstu, ovaj rad potvrđuje da budućnost računalnog vida leži u pametnijem dizajnu, a ne samo u sirovoj procesorskoj snazi. I MobileNet i EfficientNet pokazali su se kao izvediva rješenja za primjenu u stvarnom svijetu, s vremenima treniranja koja su prihvatljiva čak i na standardnom hardveru. Uloga inženjera strojnog učenja je evoluirala; ona više ne podrazumijeva samo dizajniranje slojeva, već precizno inženjerstvo podataka i prilagodbu okoline modela, što se pokazalo kao presudan faktor za uspjeh u ovom istraživanju.

Poveznica na Overleaf projekt

https://www.overleaf.com/read/xrhnsxdkppgb#401bea

Popis literature

[1] F. Chollet, *Deep Learning with Python*. Shelter Island, NY: Manning Publications Co., 2017., 384 str., ISBN: 978-1-61729-443-3.

[2] tnodecode. „The Beginning of Convolutional Neural Networks," Medium, pogledano 27. listopada 2023. adresa: https://medium.com/@tnodecode/the-beginning-of-convolutional-neural-networks-ec7dad769f8c.

[3] M. Yawar. „MobileNet." Naukri Code360, Accessed: 2026-01-23. adresa: https://www.naukri.com/code360/library/mobilenet.

[4] A. G. Howard i dr., „MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications," *arXiv preprint*, 2017., arXiv:1704.04861 [cs.CV], Accessed: 2026-01-23.

[5] A. Polukhin. „MobileNet Architectures: Overview of the MobileNet v1, MobileNet v2, MobileNet v3." Medium, Accessed: 2026-01-23. adresa: https://medium.com/@pandrii000/mobilenet-architectures-17fe7406d794.

[6] P. Baheti. „A Comprehensive Guide to Convolutional Neural Networks." V7 Labs Blog, Accessed: 2026-01-23. adresa: https://www.v7labs.com/blog/convolutional-neural-networks-guide.

[7] K. Džomba, „Konvolucijske neuronske mreže," Diplomski rad, Accessed: 2026-01-23, mag. rad, Prirodoslovno-matematički fakultet, Matematički odsjek, Sveučilište u Zagrebu, Zagreb, Croatia, 2018.

[8] „Conv2D layer — Keras API Documentation." Accessed: 2026-01-23, Keras Team. adresa: https://keras.io/api/layers/convolution_layers/convolution2d/.

[9] „MaxPooling2D layer — Keras API Documentation." Accessed: 2026-01-23, Keras Team. adresa: https://keras.io/api/layers/pooling_layers/max_pooling2d/.

[10] „Flatten layer — Keras API Documentation." Accessed: 2026-01-23, Keras Team. adresa: https://keras.io/api/layers/reshaping_layers/flatten/.

[11] „Dense layer — Keras API Documentation." Accessed: 2026-01-23, Keras Team. adresa: https://keras.io/api/layers/core_layers/dense/.

[12] A. Weirich. „Finetuning TensorFlow/Keras Networks: Basics Using MobileNetV2 as an Example." Medium, Accessed: 2026-01-23. adresa: https://medium.com/@alfred.weirich/finetuning-tensorflow-keras-networks-basics-using-mobilenetv2-as-an-example-8274859dc232.

[13] DeepLearningAI. „Mobilenets (c4w2l07),” YouTube, pogledano 27. listopada 2023. adresa: https://www.youtube.com/watch?v=GOxRSefbBoI.

[14] A. Singla. „Efficientnet Explained! | State of the Art (SOTA) in Image Classification,” YouTube, pogledano 27. listopada 2023. adresa: https://www.youtube.com/watch?v=RwQ-5v-kIck.

Popis slika

1. Prikaz točnosti po epohama koda sa laboratorijskih vježbi 11
2. Točnost modela sa laboratorijskih vježbi 11
3. Analiza modela sa laboratorijskih vježbi po klasama 11
4. Rezultati treniranja MobilenetV2 arhitekture po epohama 12
5. Analiza modela s MobileNetV2 arhitekturom po klasama 12
6. Prikaz epoha nakon povećanja slika za arhitekturu MobileNetV2 13
7. Analiza modela s MobileNetV2 arhitekturom po klasama nakon povećanja slika . 13
8. Primjena preprocess_input funkcije 14
9. Rezultati treniranja po epohama (EfficientNetB1) 14
10. Klasifikacijski izvještaj uspješnog modela 15
11. Kod s ručnom normalizacijom (dijeljenje s 255) 15
12. Rezultati neuspješnog treniranja 16
13. Klasifikacijski izvještaj neuspješnog modela 16